



École Polytechnique

Project Report, CSE305: Concurrent and Distributed Programming

Parallel Falling Sand Simulation in C++

Authors:

Ayoub Agouzoul and Rachid Tazi, École Polytechnique

Advisors:

Gleb Pogudin, Augustin Albert, Ricardo Buring, École Polytechnique

Academic year 2025/2026

Contents

1	Introduction	4
1.1	What is a falling sand game?	4
1.2	Why is falling sand interesting for concurrency?	4
1.3	Project objectives	4
1.4	Tools: Godot, GDExtension, and compute shaders	5
1.5	Constraints and evaluation criteria	5
2	Project Setup and Code Structure	5
2.1	Starting point	5
2.2	Main components	5
3	Simulation Model	6
3.1	Grid representation	6
3.2	Sequential update rule	6
3.3	Double buffering	6
3.4	Fairness	7
3.5	Conservation	7
4	CPU Implementation	7
4.1	Sequential baseline	7
4.2	Data structure considerations	7
4.3	Parallel CPU version using locks	7
5	GPU Implementation	9
5.1	Godot-GPU integration	9
5.2	Compute Shader implementation	9
6	Advanced Simulation Approaches	9
6.1	Block cellular automata	9
6.2	Margolus neighborhoods	9
6.3	Comparison of methods and expected results	10
7	Correctness Testing	11
7.1	Conservation of matter	11
7.2	Fairness test	11
8	Benchmarks and Results	12
8.1	Benchmark scenarios	12
8.2	Measured quantities	13
8.3	Measurement protocol	13
8.4	Hardware	14
8.5	Overall comparison	14
8.6	CPU thread scaling	14
8.7	GPU benchmarking	15
9	Work Repartition	16
10	Acknowledgements	16

11 Conclusion	16
12 References	18

1 Introduction

1.1 What is a falling sand game?

A falling sand game is a sandbox simulation in which the world is represented as a grid of cells. Each cell is either empty or contains a particle, such as sand. The simulation evolves in discrete time steps. At each step, each cell is updated according to simple local rules: for example, a sand particle moves downward if the cell below it is empty, or diagonally downward if the direct downward cell is blocked. Despite the simplicity of these rules, the global behavior can look realistic: grains fall, accumulate, form piles, and create slopes. This cellular automaton view is the basis of many falling sand video games [6, 1] such as in Figure 1.

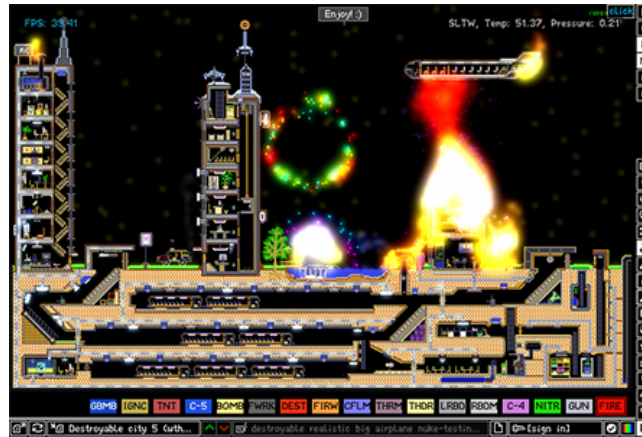


Figure 1: Screenshot from the *Powder Toy* game.

1.2 Why is falling sand interesting for concurrency?

As part of this Concurrency course, implementing a falling sand game is really interesting because the game can contain thousands or millions of cells that could be updated in parallel. Even though the local rules are very simple, concurrency is not trivial because neighboring cells depend on one another. Conflicts must be managed: two particles may want to move into the same cell, or a particle may move into a cell that another has just left.

Many other issues, such as bias towards a single direction when grains fall, make this project an interesting case study. Indeed, a sequential algorithm is easier to reason about, because the update order is clear. A parallel algorithm may be much faster, but requires a strategy to avoid conflicting writes and to define what it means for one simulation step to be correct. The project therefore connects directly to the main themes of the concurrency course: data dependencies, synchronization, data races, work decomposition, and performance measurement.

1.3 Project objectives

The objective of the project is to implement and evaluate several versions of a falling sand simulation:

1. a sequential CPU implementation in C++;
2. a parallel CPU implementation using multiple threads;
3. a GPU implementation using compute shaders;
4. correctness tests for conservation of matter and fairness;
5. benchmarks comparing runtime and speedup across grid sizes, thread counts, and GPU workgroup sizes.

The code is available at:

<https://github.com/Rach0o0/Falling-sand-game>

1.4 Tools: Godot, GDExtension, and compute shaders

We use Godot as the game engine and visualization framework [3]. Godot displays the falling sand grid as an image or texture, updated at each frame from data produced by the simulation. This allows us to separate visualization from the main computation.

The simulation logic is implemented in C++ using GDExtension [2]. GDExtension allows C++ classes to interact with the Godot engine without modifying and recompiling the full engine. In practice, our code exposes a C++ class to Godot, so that the scene can create and update a sand grid object while the simulation itself remains in C++.

For the GPU version, we use compute shaders written in GLSL. A compute shader is a program executed by the GPU for general computations instead of only graphics rendering. Godot supports compute shaders and provides an interface for dispatching GPU workgroups [4]. This is well suited to a cellular automaton, because each GPU invocation can be responsible for one cell or for a small group of cells, allowing for a huge speedup in theory.

1.5 Constraints and evaluation criteria

We evaluate the project following two criteria:

1. **Correctness:** conservation of the number of particles, fairness of left/right decisions, and stability of final states.
2. **Performance:** execution time per frame, total runtime for benchmark scenarios, and speedup compared to the sequential baseline.

2 Project Setup and Code Structure

2.1 Starting point

Since this was our first project using Godot, the first step was to understand how to connect a C++ simulation with the engine. We used the course material from “TD3 in Godot” and the “Godot implementation of Game of Life” as starting points. These examples helped us install the required tools, including the `godot-cpp` bindings and `scons`, which is used to compile Godot C++ projects.

In Godot, C++ classes must be registered so that the engine can create them and expose their methods to the editor and to GDScript. This is done in a file such as `register_types.cpp`, which defines initialization and termination functions for the extension. During initialization, the extension registers the C++ classes that Godot should know about.

Because the purpose of the project is concurrency concepts in C++ rather than learning every detail of Godot from scratch, we used an LLM to convert the example code given by one of the course TAs as well as some of the Godot documentation examples, from GDScript and C# to C++ to get a working foundation for the Godot/GDExtension compute shader pipeline. We then focused our own implementation effort on the sand simulation, correctness tests, parallelization, and benchmarking.

2.2 Main components

The base of the code is the class `SandGrid` which extends the `Sprite2D` Godot object. This class represents the grid of cells and provides the methods used to initialize, update, display, and benchmark the simulation. The project is organized around the following components:

Component	Role
register_types.cpp	Registers the C++ extension classes so that Godot can instantiate and call them.
sand_grid.*	Defines the grid, cell representation, update rules, texture update, and simulation methods.
benchmark.*	Runs benchmark scenarios and records runtimes, speedups, and correctness statistics.
*.glsl	Contains the GPU update kernel for the compute-shader implementation.
README.md	Explains how to build and run the project on the target machines.

3 Simulation Model

3.1 Grid representation

The simulation world is a two-dimensional grid of width W and height H . Each position (x, y) stores one cell value. In the current version, a cell can have one of two states:

1. EMPTY: no particle is present;
2. SAND: a sand particle is present.

A simple dense representation is an array of size $W \times H$. The index of cell (x, y) can be computed as

$$\text{index}(x, y) = y \cdot W + x.$$

3.2 Sequential update rule

For each simulation step, sand particles are updated according to gravity. The rule for one sand particle is:

1. If the cell directly below is empty, move the particle down.
2. Otherwise, if one of the two diagonal cells below is empty, move the particle diagonally down.
3. If both diagonal cells are possible, choose left or right in an unbiased way (by simulating a coin flip).
4. If no move is possible, the particle stays in place.

We quickly noticed that the order in which the grid is traversed matters. If we update cells from top to bottom, a particle may move down and then be processed again in the same step. To avoid this, the sequential version should traverse from bottom to top, so that a particle can move at most once per step.

A simplified sequential algorithm is shown below:

```
for y = height - 2 down to 0:
  for x = 0 to width - 1:
    if grid[x, y] == SAND:
      update_sand_particle(x, y)
```

The bottom row does not need to be updated because particles there cannot move further down.

3.3 Double buffering

One possible implementation uses double buffering. Instead of modifying the current grid directly, the algorithm reads from an input grid and writes the next state to an output grid. At the end of the step, the two buffers are swapped.

Double buffering makes the notion of a time step clearer: all decisions are based on the same old state. However, it does not automatically solve all conflicts. For example, two particles may still attempt to write to the same destination cell in the output buffer. Therefore, even with double buffering, we needed to resolve

conflicts using some of the concepts we learned, such as atomic operations and `compare_exchange()` (CPU parallel version below).

3.4 Fairness

A fair simulation must not systematically prefer one side, otherwise piles lean and slopes look unnatural. The main risk is the diagonal tie-break: when both down-left and down-right are free, a fixed preference biases every such particle in the same direction. We therefore decide the direction with an unbiased coin flip drawn from a pseudo-random generator (`std::mt19937`). An earlier version derived the choice from the cell position, which is cheap but produces a static, visible pattern instead of true randomness. In the parallel versions, each thread owns its own generator, so the simulation stays fair without sharing RNG state between threads.

3.5 Conservation

The number of sand particles should remain constant unless particles are intentionally added or removed by user interaction (which is not the case for now, as we haven't added such particle types). Our conservation test computes

$$N_t = \sum_{x=0}^{W-1} \sum_{y=0}^{H-1} \mathbf{1}_{\text{grid}_t(x,y)=\text{SAND}}$$

for the initial grid and for the last frame. We then check that $N_t = N_0$.

This test is especially important for parallel versions, because race conditions may duplicate or destroy particles.

4 CPU Implementation

4.1 Sequential baseline

The sequential CPU version is the reference implementation. Its main purpose is not only to provide a working simulation, but also to define the expected behavior for correctness tests and speedup comparisons.

The baseline algorithm iterates over all cells. For each sand cell, it checks at most three neighbors: down, down to the left, and down to the right. Therefore, the time complexity of one step is $O(W \cdot H)$.

4.2 Data structure considerations

One of the early ideas we had was about the data structure we use to represent the cells in our simulation. We thought about a sparse representation since only a small fraction of the grid contains sand. However, falling sand requires frequent checks of neighboring cells. For this reason, a dense grid is often simpler and faster, because checking neighbors is just array indexing. A sparse matrix could reduce memory usage for very empty grids, but it would complicate neighbor queries and conflict detection.

For the first implementation, we use a dense array (`std::vector<uint8_t>`) of cells.

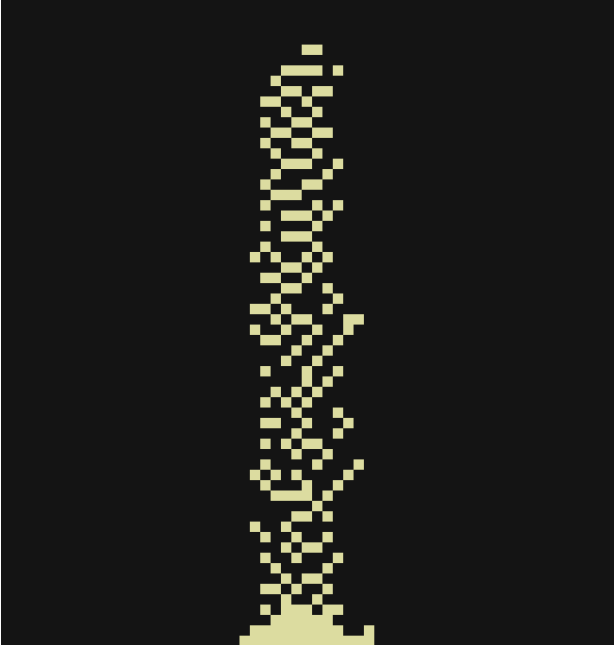
4.3 Parallel CPU version using locks

The cost of one step is $O(W \cdot H)$, and every cell is treated independently apart from local neighbor interactions, which makes the step a natural target for parallelism. In both parallel backends we partition the grid into bands and assign one `std::thread` to each band per step and the threads are joined at the end. The difficulty is not the partitioning itself but the conflicts at the boundaries, where two particles may compete for the same destination cell: if unhandled, this duplicates or destroys sand and breaks conservation. We implemented two strategies that resolve this differently.

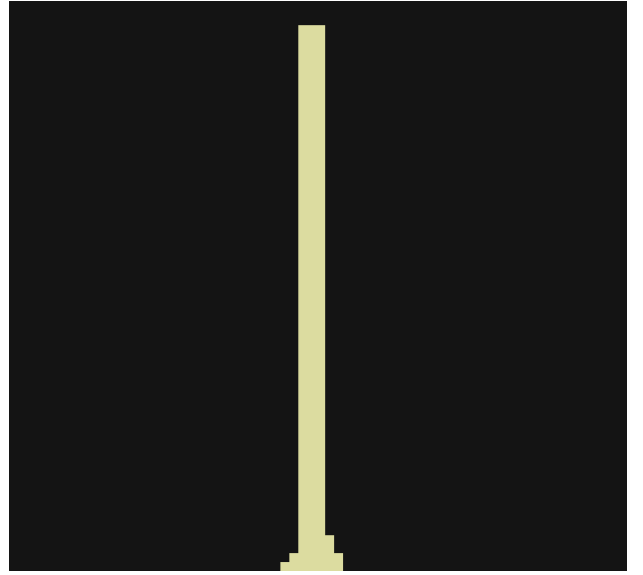
The first backend (CPU Parallel) stores the grid as two buffers of `std::vector<std::atomic<uint8_t>>` and splits the rows into horizontal bands. Each thread reads from the frozen old buffer and writes the new one, so threads never observe each other's writes. Two particles can still target the same empty destination; we resolve this with an atomic `compare_exchange` on that cell in the output buffer, so exactly one particle wins and the other stays in place. The result is lock-free and conservative. However, because every particle decides from the same frozen snapshot, a fully packed column cannot fall as a block: each grain still sees the one below it as present and spills sideways. The model therefore behaves as a synchronous cellular automaton (as opposed to an asynchronous one), which is correct and fair but visually different from the in-place (i.e. no double buffer) sequential rule.

To reproduce the sequential behavior, the second backend (CPU Parallel ColumnBand) keeps a single buffer and applies the same bottom-to-top in-place sweep, but splits the grid into vertical bands of *columns*. In-place updates let a column fall as a unit, exactly as in the sequential version. Here the only shared data are the two columns on each side of a band boundary, since a particle can spill diagonally across it. We guard each boundary (seam) with its own `std::mutex`, taken only when a particle sits within one column of an edge. All other (interior) particles run lock-free. This keeps cross-seam moves both race-free and conservative, and with a single thread the output is identical to the sequential baseline. It thus serves as a correct in-place reference against which more advanced methods (such as a block cellular automaton) can be compared.

Figure 2 shows the two models on the same input: the double buffering method spreads the grains sideways, while the in-place column bands keep the column falling vertically like the sequential rule.



(a) Double buffering + atomic CAS. The old buffer frozen makes the packed column scatter sideways.



(b) In-place column bands. The column falls correctly, matching CPU sequential.

Figure 2: Same packed-column initial state under the two parallel CPU backends.

5 GPU Implementation

5.1 Godot-GPU integration

To use compute shaders, we relied on the compute shader support provided by the Godot engine. As a starting point, we used a “Game of Life” example provided on Moodle by the professors. This example demonstrated how to connect Godot with a compute shader, but was implemented in GDScript. Since our project is developed in C++, we had to translate and adapt the Godot API calls from GDScript to their C++ equivalents. As advised by the professor, we used an LLM to identify these C++ API calls and generated a version of the code. Then we use a GPU backend written in C++, which communicates with a GLSL compute shader responsible for updating the simulation state in parallel.

5.2 Compute Shader implementation

The GLSL file contains the compute shader. It performs one simulation step of the falling sand. As with the parallel method on the CPU, we use double buffering. The grid is represented by one texture containing the current state, and an output texture into which the next state is written. Each GPU thread is responsible for one cell of the grid.

To determine which data to use, layout declarations are employed so that the shader can obtain this information. These declarations specify which textures contain the current and next states of the grid, as well as how the GPU threads are grouped (workgroups). For example, the statement `layout(local_size_x = 16, local_size_y = 16)` indicates that each workgroup contains 16×16 GPU threads.

For each grain of sand, the physical rules are applied to determine the desired destination using the `wanted_target()` function. If no movement is possible, the grain remains in place. To avoid a systematic bias toward one side when both diagonal movements are possible, the shader uses the `hash_uvec2()` function, which generates a pseudo-random value based on a cell’s coordinates.

To avoid conflicts, the `accepted_source_for()` function allows each target cell to examine neighboring grains that might enter it and choose which one is allowed to occupy that position in the next state. This approach ensures that a cell can receive only one grain at a time.

6 Advanced Simulation Approaches

6.1 Block cellular automata

The previous GPU implementation had some limitations. Since multiple particles can compete for the same destination cell, it was necessary to add additional logic to resolve conflicts and ensure mass conservation.

To optimize this, we implemented a block-based cellular automaton [5]. Instead of updating each cell individually, the grid is divided into small 2×2 blocks that are updated as a whole. Each block is processed independently of its neighbors during a simulation step, which allows us to avoid write conflicts between threads. A single thread is responsible for updating a block of cells. Thus, this method reduces the need for conflict resolution strategies.

6.2 Margolus neighborhoods

Our block-based cellular automaton is based on Margolus’s neighborhood rules. The grid is divided into independent 2×2 blocks:

$$\begin{array}{cc} a & b \\ c & d \end{array}$$

Each GPU invocation updates an entire block. Within a block, simple local rules are applied. A grain of sand can move downward if the cell below it is empty. If downward movement is not possible, a diagonal movement can be made within the block.

To allow the grains to interact with cells in other blocks, the block partition is shifted by one cell horizontally and vertically at each step of the simulation. We therefore alternate between two phases:

- Phase 0: the blocks start at the coordinates $(0, 0)$, $(2, 0)$, $(4, 0)$, ...
- Phase 1: the blocks start at coordinates $(-1, -1)$, $(1, -1)$, $(3, -1)$, ...

This allows the particles to gradually interact with neighboring blocks and prevents them from becoming trapped inside a single block.

The advantage of this method is that no two GPU invocations write to the same cell during a given step. This makes the algorithm naturally parallel and well-suited for execution on a GPU.

However, the Margolus method can introduce a local directional bias. During a given phase, a particle can only move within its current 2×2 block. Therefore, while a grain of sand could theoretically move in both diagonal directions on the full grid, only one of these directions may be available within the current block. This bias is, nevertheless, reduced by the alternating phases.

6.3 Comparison of methods and expected results

Before presenting empirical measurements, we summarize the implementations we built, how they differ, and what we expect from each one. We have two families of methods, distinguished by where the work runs (CPU threads or the GPU) and by the underlying physical model (a synchronous, double-buffered cellular automaton versus an in-place or block-based one).

CPU: Both CPU-parallel backends partition the grid into bands and assign one `std::thread` per band per step. They differ in how they handle the conflicts that arise at band boundaries, where two grains may compete for the same destination cell. The *column-band* backend keeps a single buffer and applies the same bottom-to-top in-place sweep as the sequential baseline, so a packed column falls as a unit. Its only shared data are the two columns straddling each band boundary. We guard each boundary with its own `std::mutex`, taken only by grains within one column of an edge, while interior grains run lock-free. At one thread it is identical to the sequential reference, which makes it a correct in-place baseline. The *Margolus* backend instead splits the grid into independent 2×2 blocks: because no two threads ever touch the same cell, conservation holds by construction and no locks are needed at all, at the cost of being a genuinely different physical model that no longer matches the sequential rule pixel-for-pixel.

	Column bands	Margolus blocks
Update unit	single cell, in-place sweep	independent 2×2 block
Shared data	two columns per band seam	none (disjoint blocks)
Synchronization	one <code>std::mutex</code> per seam	none
Conservation	guaranteed by the seam locks	guaranteed by construction
Steps per frame	one sweep	two (offset 0, then $(1, 1)$)
Versus sequential	identical at one thread	different physical model

GPU: The same opposition reappears on the GPU. The per-cell compute shader assigns one thread to each cell and double-buffers: since a thread may only write its own cell, a move requires the source and destination cells to *agree* through a check (each computes its wanted target or accepted source), which avoids conflicts without atomics but recomputes neighbor decisions redundantly and behaves as a synchronous cellular automaton (the

same sideways scatter on packed columns as the double-buffer CPU version). The GPU Margolus shader assigns one thread per 2×2 block, reads four cells, applies the local gravity rule, and writes four cells, in two phases per step with an alternating block offset. No check is required, conservation is automatic, and each thread does a fixed, small amount of work.

	Double buffer per-cell shader	Margolus shader
Thread granularity	one thread per cell	one thread per 2×2 block
Conflict avoidance	source/target handshake	none (disjoint blocks)
Reads per thread	many and redundant	exactly four
Dispatches per frame	one	two (one per phase)
Conservation	relies on a symmetric handshake	by construction
Physical model	synchronous CA (scatter)	block CA (columns fall coherently)

Expected results. On correctness, we expect every backend to pass the conservation test. Only the CPU sequential and CPU column-band versions should reproduce the reference dynamics exactly; the double buffered and Margolus models are correct but visually distinct. On performance, we expect the parallel backends to improve with grid size and to lose to the sequential baseline on small grids, where thread creation dominates at each step. Among the parallel methods, the lock-free block CA should scale better than the seam-locked column bands, as it removes all synchronization on the hot path. On the GPU, the Margolus shader should outperform the per-cell shader by doing far less redundant work per thread, despite issuing two dispatches per step. The empirical sections that follow test these expectations.

7 Correctness Testing

7.1 Conservation of matter

The first correctness test counts the number of sand particles before and after a fixed number of frames. For a simulation without user interaction, the count should remain constant. We run this check on every scenario, for all six implementations, and for all grid sizes and repeats of the benchmark sweep. The table below reports the counts for a representative 256×256 grid:

Scenario	Initial particles	Final particles	Pass/Fail
Single drop	1088	1088	Pass
Hourglass	15038	15038	Pass
Checkerboard	32768	32768	Pass
Random dense grid	32634	32634	Pass
Full upper half	32768	32768	Pass

No implementation lost or duplicated a single grain. As expected, the block-CA methods (CPU and GPU Margolus) cannot break conservation by construction, since disjoint blocks never write the same cell; the lock-based and double-buffer methods also pass, confirming that the seam locks and the `compare_exchange` conflict resolution are correct.

7.2 Fairness test

To test fairness we drop a single symmetric source of sand from the top centre onto an empty floor, let it settle, and count how many grains end up to the left and to the right of the source column. For a perfectly symmetric setup an unbiased simulation should give $L \approx R$. This measurement is done by only reading the grid at the final step, and so applies uniformly to every method. The table below reports the settled counts on a 256×256 grid.

Implementation	Left L	Right R	Bias $B = \frac{ L-R }{L+R}$
CPU sequential	325	737	0.388
CPU double buffer	289	775	0.457
CPU column bands	479	578	0.094
CPU Margolus	528	528	0.000
GPU per-cell	505	551	0.044
GPU Margolus	525	526	0.001

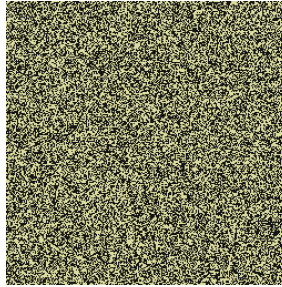
Two effects show up here. First, the coin flip itself is unbiased: this is visible in the GPU per-cell shader ($B = 0.04$), which decides the diagonal direction from a position hash rather than a counter. Second, and larger, is a *scan-order* effect in the cpu sequential method: because each row is swept left to right, a grain that takes its down-right cell removes that option from the grain on its right, which then drifts further right. This pushes the whole pile to the right and explains the large bias of the sequential and double buffer backends ($B \approx 0.39$ – 0.46). The column-band backend reduces it ($B = 0.09$) because the sweep is restarted independently in each of the bands (equal to the number of CPU threads used), so the drift is somewhat local and cancels out. The Margolus backends are symmetric essentially by construction ($B \leq 0.001$). Although the block rule is deterministic, the left/right symmetry comes from the alternating phase offset. The practical conclusion is that the residual bias is dominated by traversal order, which is inherent to methods such as the CPU sequential version. A solution could be to alternate the traversal direction every frame, trying to replicate the alternating phases of the Margolus method.

8 Benchmarks and Results

8.1 Benchmark scenarios

We evaluate the implementations on four initial grid states that stress different aspects of the update, plus a fifth state used only for the fairness test:

1. **Random dense:** every cell is filled with sand with probability 0.5 (fixed seed). Uniform movement across the whole grid.



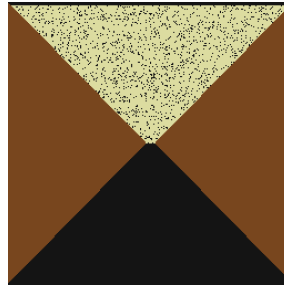
2. **Checkerboard:** sand on every cell where $(x + y)$ is even, this tests the update rule's robustness as the maximal number of grains can move on the first frame.



3. **Full upper half:** the top half of the grid is completely packed with sand, to observe how the sand settles at a large scale.



4. **Hourglass:** wood funnel walls converging to a narrow neck, with the upper chamber nearly full of sand. Grains collide and stream through the bottleneck, allowing us to observe multiple types of movement over many frames.



5. **Single source:** a small symmetric column of sand dropped from the top centre onto an empty floor. Used only for the fairness test above.



The four first scenarios are run for throughput measurement for every method and grid size, and the results are averaged over each simulation method.

8.2 Measured quantities

For each benchmark we record: grid size $W \times H$, number of particles, number of simulated steps, total runtime, average time per step, throughput in millions of cells updated per second (Mcells/s), speedup relative to the sequential CPU baseline, the final particle count (for conservation), and the fairness metric when relevant. Every individual run is written to a CSV file using the GDScript `benchmark.gd` script so that all tables and plots below can be produced using a separate Python script.

8.3 Measurement protocol

All timings come from a headless benchmark entry that builds the backend, loads a scenario, and times the step loop: setup, the initial CPU→GPU upload, and the display texture are excluded. For the GPU backends a private `RenderingDevice` is synchronised (`submit` then `sync`) after each step, so the measurement reflects real GPU work and not just command submission.

To reduce variance we run one untimed warm-up pass (shader compilation, cache warming,...) followed by three timed runs from the exact same initial state, and report the median. Each configuration uses a fixed seed and a fixed number of steps, and the four throughput scenarios are averaged. The whole sweep runs in

a single pass under the Vulkan driver. Finally, we time one engine `step()`, and one CPU Margolus step runs both Margolus phases (a full frame) whereas one GPU Margolus step runs a single phase in our current implementation, which is why we keep Mcells/s as the primary metric since it stays comparable across all methods.

8.4 Hardware

Hardware used to collect the benchmark data.

Item	Value
CPU	AMD Ryzen AI 9 HX 370
Number of cores	12 cores / 24 threads
RAM	32 GB LPDDR5
GPU	NVIDIA GeForce RTX 4060 Laptop (100W max TGP)
Operating system	CachyOS (Arch Linux), kernel 7.0.10
Compiler	GCC 16.1.1
Godot version	4.6.3 stable

8.5 Overall comparison

Figure 3 shows the throughput of all six backends as a function of grid size, averaged over the four scenarios. It mostly matches our expectations (Section 6.3). On small grids the sequential CPU baseline is the fastest single option: it has no thread-creation or synchronisation overhead, and at 128×128 the parallel CPU backends are an order of magnitude slower because spawning and joining threads each step dominates the tiny workload. As the grid grows the cost of the sequential sweep grows as $O(W \cdot H)$ and it is overtaken: by 2048×2048 all three parallel CPU backends are faster than sequential, with CPU Margolus the best of them. The GPU backends dominate at every size from 256×256 on, reaching roughly 9 Gcells/s and a $17\times$ speedup over the sequential baseline at 2048×2048 . The two GPU variants are very close, and the two block-CA methods are never slower than their per-cell/locking counterparts, consistent with the prediction that removing synchronisation helps.

8.6 CPU thread scaling

We measure runtime as a function of the number of threads on the largest grid (2048×2048), where the per-step work is large enough to amortise thread management. The table below reports the in-place column-band backend; Figure 4 compares the speedup of all three parallel CPU backends.

Threads	Runtime (ms)	Time/step (ms)	Speedup
1	1353.4	13.53	1.00
2	750.5	7.50	1.80
4	431.0	4.31	3.14
8	246.0	2.46	5.50
12	238.1	2.38	5.69
24	217.7	2.18	6.22

The speedup is good up to the physical core count and then flattens. The machine has 12 physical cores and 24 SMT threads: scaling is close to linear up to 4 threads, keeps improving to about 8, and gains little beyond 12, where the simultaneous multithreading starts sharing execution across cores. The remaining gap to ideal scaling is the expected combination of memory bandwidth limits, thread creation and join every step, and, for the column-band backend, the per-seam locking at band edges. Among the three, CPU Margolus reaches the

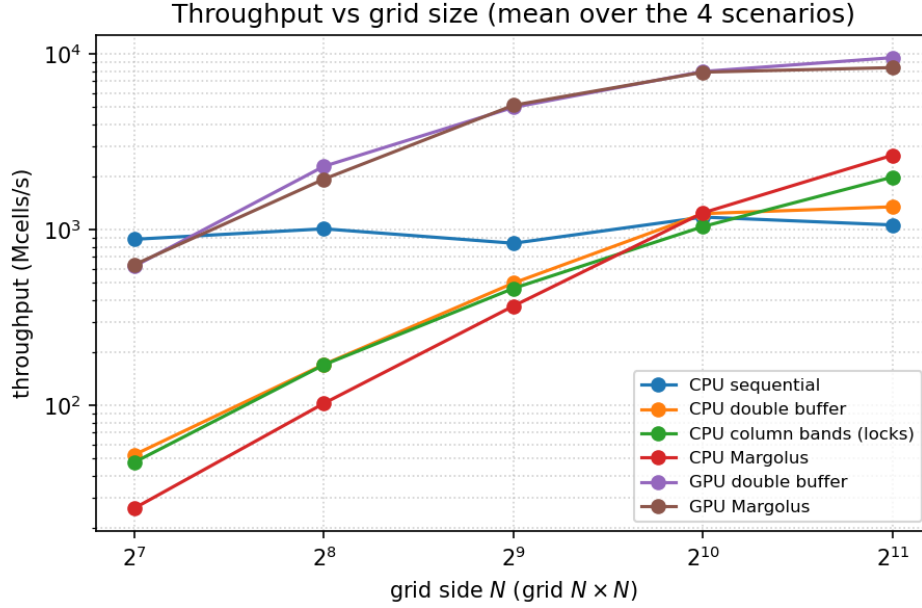


Figure 3: Throughput (Mcells/s, log–log) versus grid side N for a $N \times N$ grid, averaged over the four scenarios. Higher is better.

best absolute time (it is lock-free and updates in place), the column bands are close behind, and the double-buffer backend is slowest in absolute terms because it pays for atomics and a full second buffer, even though its *relative* speedup looks similar.

8.7 GPU benchmarking

For the GPU version we compare grid sizes and the compute-shader workgroup size (the `local_size` of the kernel). The workgroup size is substituted into the shader at compile time, so the same kernel is benchmarked with several layouts. The table below reports, for the per-cell shader, the best workgroup at each grid size together with the resulting time per step and the speedup over the sequential CPU baseline; Figure 5 shows the full workgroup sweep.

Grid size	Best workgroup	Time/step (ms)	Speedup
256×256	8×8	0.030	$2.8\times$
512×512	8×8	0.047	$8.8\times$
1024×1024	8×32	0.126	$15.9\times$
2048×2048	32×8	0.487	$17.6\times$

The GPU speedup grows with grid size: small grids do not have enough cells to fill the GPU and are dominated by per-dispatch overhead, while at 2048×2048 the device is well occupied and the per-cell shader reaches about $17.6\times$ the sequential CPU baseline (9.6 Gcells/s). The choice of workgroup size has a real but moderate effect (from what we observed, within 10–20%): very small groups such as 8×8 are best on small grids, while larger or asymmetric groups (8×32 , 32×8) are slightly better on the large grids, maybe because they map better onto the hardware’s memory access pattern.

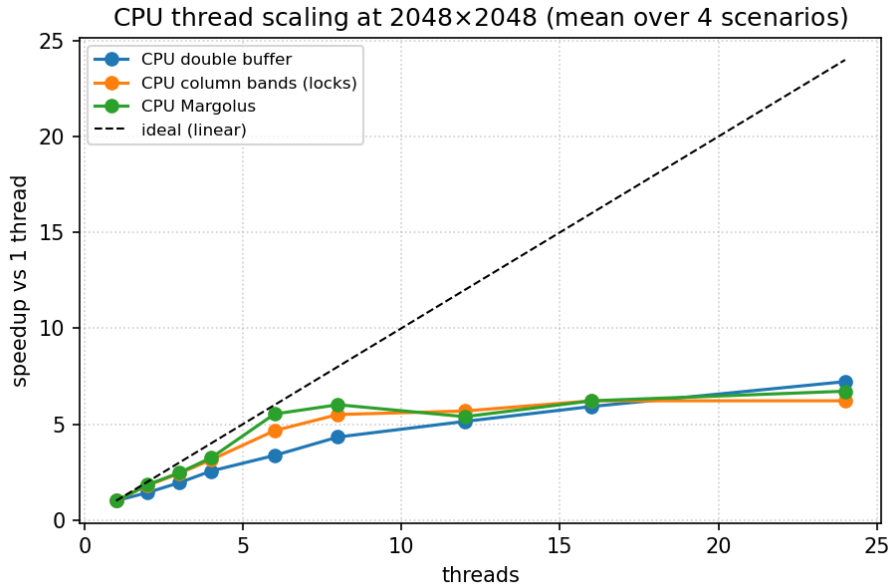


Figure 4: CPU speedup versus thread count at 2048×2048 , averaged over the four scenarios. The dashed line is ideal linear scaling.

9 Work Repartition

At the beginning of the project, we both worked on understanding the Godot setup and attempting sequential CPU implementations that were then merged and used as the baseline.

The planned repartition at the beginning of the project was:

Person	Tasks
Ayoub Agouzoul	Godot/GDExtension setup, parallel CPU implementation, correctness tests, benchmarking implementation.
Rachid Tazi	Godot compute shader setup, sequential CPU implementation, GPU compute shader implementation.
Both	Design choices, benchmark scenarios, result interpretation, final report and presentation.

10 Acknowledgements

We would like to thank the CSE305 teaching team for the project statement, the Godot Game of Life example, and their guidance on compute shaders and parallel simulation. These resources were useful for setting up the Godot/GDExtension pipeline and for understanding how to structure the GPU backend.

We also acknowledge the use of large language models as support tools for understanding parts of the Godot C++ API, debugging, and improving the wording of the report.

11 Conclusion

This project uses falling sand as a practical example of parallel simulation. The local rules are simple, but parallel execution introduces non-trivial correctness issues: write conflicts at partition boundaries, conservation

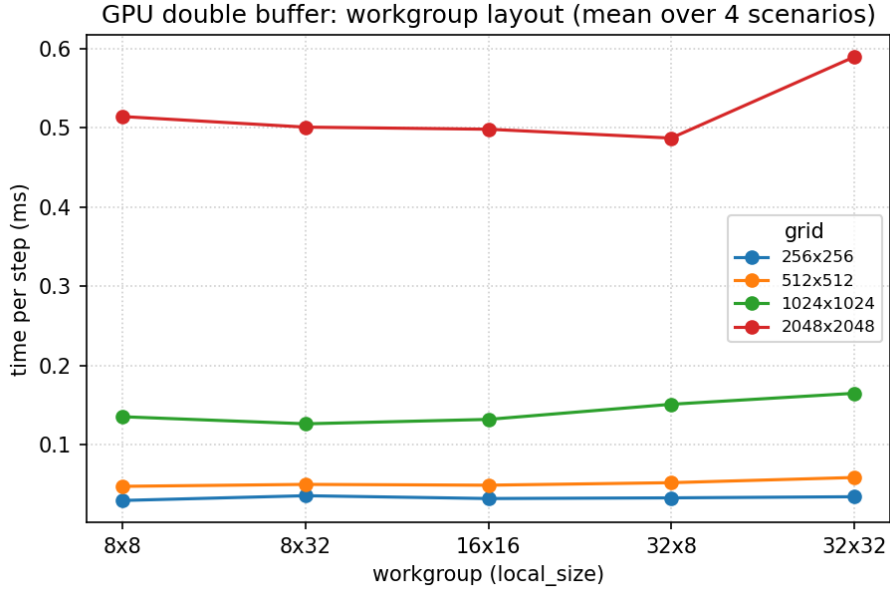


Figure 5: Per-cell GPU shader: time per step for several workgroup layouts and grid sizes (mean over the four scenarios).

of particles, and directional fairness. We addressed these with three CPU strategies (a double-buffer push model with atomic conflict resolution, in-place column bands with locks for conflict resolution, and a lock-free Margolus block automaton) and two GPU compute shaders (per-cell double buffer and Margolus).

Across all scenarios every implementation conserves matter, and the benchmark sweep gives a clear ordering: the sequential baseline wins on small grids but is overtaken once the grid is large enough to amortise parallel overhead, the CPU backends reach roughly a $6\text{--}7\times$ speedup on 24 threads (best for the lock-free Margolus version), and the GPU reaches about $17\times$ the sequential baseline at 2048×2048 . The fairness study showed that the main source of bias is the fixed traversal order, which the block CA methods avoid by construction. Overall, the block cellular automaton offers the best balance of speed, scalability and correctness on both CPU and GPU, while the column-band backend remains the closest reproduction of the sequential rule from a dynamics point of view. Full code with instructions available at: The code is available at:

<https://github.com/Rach0o0/Falling-sand-game>

12 References

- [1] Jonathan Devlin and Micah D. Schuster. Probabilistic cellular automata for granular media in video games. *The Computer Games Journal*, 10(1):111–120, 2021.
- [2] Godot Engine documentation. Gdextension. <https://docs.godotengine.org/en/4.4/tutorials/scripting/gdextension/index.html>, 2026.
- [3] Godot Engine documentation. Godot engine documentation. <https://docs.godotengine.org/en/stable/index.html>, 2026.
- [4] Godot Engine documentation. Using compute shaders. https://docs.godotengine.org/en/latest/tutorials/shaders/compute_shaders.html, 2026.
- [5] Wikipedia contributors. Block cellular automaton. https://en.wikipedia.org/wiki/Block_cellular_automaton, 2025.
- [6] Wikipedia contributors. Falling-sand game. https://en.wikipedia.org/wiki/Falling-sand_game, 2025.